Ejercicio 1: Tipo Articulo (0,5 ptos)

```
public record Articulo(String titulo, String categoria, Integer paginas) {

	public static final Integer MINIMO_PAGINAS = 10;

	public Articulo {
		Checkers.check("El título no puede estar vacío",
				!titulo.isEmpty());
		Checkers.check("El número de páginas debe ser > 0",
				paginas > 0);
	}

	public Boolean articuloLargo () {
		Boolean res = false;
		if (paginas >= MINIMO_PAGINAS) {
			res = true;
		}
		return res;
	}
}
```

Ejercicio 2: tipo Revista (2 ptos)

```
public class Revista implements Comparable<Revista> {

	private String numero;
	private LocalDate fechaPublicacion;
	private Integer ejemplares;
	private Integer paginasFijas;
	private Integer anuncios;
	private List<Articulo> articulos;

	public Revista(String numero, LocalDate fechaPublicacion,
			Integer ejemplares, Integer paginasFijas,
			Integer anuncios, List<Articulo> articulos) {

		Checkers.check("Debe haber al menos un artículo",
				articulos.size() >= 1);
		Checkers.check("La fecha de publicación debe ser un lunes",
				fechaPublicacion.getDayOfWeek().equals(DayOfWeek.MONDAY));

		this.numero = numero;
		this.fechaPublicacion = fechaPublicacion;
		this.ejemplares = ejemplares;
		this.paginasFijas = paginasFijas;
		this.anuncios = anuncios;
		this.articulos = new ArrayList<>(articulos);

		Checkers.check("El número de páginas debe ser múltiplo de 8",
				getTotalPaginas() % 8 == 0);
	}

	public Integer getEjemplares() {
		return ejemplares;
	}
```

```
public void setEjemplares(Integer ejemplares) {
	this.ejemplares = ejemplares;
}

public String getNumero() {
	return numero;
}

public LocalDate getFechaPublicacion() {
	return fechaPublicacion;
}

public void setFechaPublicacion(LocalDate fechaPublicacion) {
	Checkers.check("La fecha de publicación debe ser un lunes",
			fechaPublicacion.getDayOfWeek().equals(DayOfWeek.MONDAY));
	this.fechaPublicacion = fechaPublicacion;
}

public Integer getPaginasFijas() {
	return paginasFijas;
}

public Integer getAnuncios() {
	return anuncios;
}

public List<Articulo> getArticulos() {
	return new ArrayList<>(articulos);
}

public Integer getTotalPaginas() {
	Integer res = paginasFijas;
	for (Articulo a: articulos) {
		res += a.paginas();
	}
	return res;
}

public Boolean getRevistaTematica() {
	Boolean res = false;
	for (Articulo a: articulos) {
		if (a.articuloLargo()) {
			res = true;
			break;
		}
	}
	return res;
}

public Extension getExtension() {
	Extension res = Extension.BAJA;
	Integer np = getTotalPaginas();
	if (np >= 32 && np <= 64) {
		res = Extension.MEDIA;
	} else if (np > 64) {
		res = Extension.ALTA;
	}
	return res;
}
```

```
	public String toString() {
		return "Revista [numero=" + numero + ", fechaPublicacion="
				+ fechaPublicacion + ", ejemplares=" + ejemplares
				+ ", paginasFijas=" + paginasFijas + ", anuncios="
				+ anuncios + ", articulos=" + articulos + "]";
	}

	public int hashCode() {
		return Objects.hash(numero);
	}

	public boolean equals(Object obj) {
		if (this == obj)
			return true;
		if (obj == null)
			return false;
		if (getClass() != obj.getClass())
			return false;
		Revista other = (Revista) obj;
		return Objects.equals(numero, other.numero);
	}

	public int compareTo(Revista o) {
		return numero.compareTo(o.getNumero());
	}

}
```

Ejercicio 3: Factoría (1 pto)

```
	private static final String SEPARADOR_PRINCIPAL = ";";
	private static final String SEPARADOR_ARTICULOS = ",";
	private static final String SEPARADOR_ARTICULO = "-";

	public static Revista parseaRevista(String revistaStr) {

		Checkers.checkNoNull(revistaStr);
		String [] trozos = revistaStr.split(SEPARADOR_PRINCIPAL);
		Checkers.check("Formato no válido <"+revistaStr+">",
				trozos.length == 6);

		String numero = trozos[0].trim();
		LocalDate fechaPublicacion = LocalDate.parse(trozos[1].trim(),
				DateTimeFormatter.ofPattern("d/M/y"));
		Integer ejemplares = Integer.valueOf(trozos[2].trim());
		Integer paginasFijas = Integer.valueOf(trozos[3].trim());
		Integer anuncios = Integer.valueOf(trozos[4].trim());
		List<Articulo> articulos = parseaArticulos(trozos[5].trim());

		return new Revista(numero, fechaPublicacion, ejemplares,
				paginasFijas, anuncios, articulos);
	}

	private static List<Articulo> parseaArticulos(String articulosStr) {
		articulosStr = articulosStr.replace("[", "").replace("]", "");
		String [] trozos = articulosStr.split(SEPARADOR_ARTICULOS);

		List<Articulo> res = new ArrayList<>();
```

```
		for (String trozo: trozos) {
			res.add(parseaArticulo(trozo.trim()));
		}
		return res;
	}

	private static Articulo parseaArticulo(String articuloStr) {
		String [] trozos = articuloStr.split(SEPARADOR_ARTICULO);
		Checkers.check("Formato de artículo no válido <"+articuloStr+">",
				trozos.length == 3);
		String titulo = trozos[0].trim();
		String categoria = trozos[1].trim();
		Integer paginas = Integer.valueOf(trozos[2].trim());
		return new Articulo(titulo, categoria, paginas);
	}
```

Ejercicio 4: Tratamientos secuenciales (6,5 ptos)

Tipo Asociacion (0,25 puntos)

```
	public Integer getNumeroRevistas() {
		return revistas.size();
	}

	public void incorporaRevista(Revista re) {
		if (!revistas.contains(re)) {
			revistas.add(re);
		}
	}

	public LocalDate obtieneFechaRevista(Integer pos) {
		Checkers.check("La posición debe ser >= 0 y < "
			+ revistas.size(), pos >= 0 && pos < revistas.size());
		return revistas.get(pos).getFechaPublicacion();
	}
```

Tipo AsociacionUniversitaria (0,25 puntos)

```
public class AsociacionUniversitaria extends Asociacion {

	private String universidad;

	public AsociacionUniversitaria(String nombre, Integer anyo,
			String uni, List<Revista> revistas) {
		super(nombre, anyo, revistas);
		this.universidad = uni;
	}

	public String toString() {
		String s = getRevistas().stream()
				.map(Object::toString)
				.collect(Collectors.joining("\n"));
		return super.toString() + " (" + universidad + ")" +  "\n" + s;
	}
	. . .
}
```

Ejercicio 4.1 (1,5 puntos)

```
public Map<String, Integer> getTotalPaginasPorCategoria(Integer anyo) {
	Map<String, Integer> res = new HashMap<>();

	for (Revista r: getRevistas()) {
		if (anyo == null || r.getFechaPublicacion().getYear() > anyo) {

			for (Articulo a: r.getArticulos()) {
				String clave = a.categoria();
				Integer valor = a.paginas();

				if (!res.containsKey(clave)) {
					res.put(clave, valor);
				} else {
					res.put(clave, res.get(clave) + valor);
				}
			}
		}
	}
	return res;
}
```

Ejercicio 4.2 (1 punto)

```
public List<String> getTitulosOrdenados(String cadena) {
	Comparator<Articulo> cmp = Comparator.comparing(Articulo::paginas)
			.reversed().thenComparing(Articulo::categoria);
	return getRevistas().stream()
			.flatMap(r -> r.getArticulos().stream())
			.filter(a -> a.titulo().toUpperCase()
				.contains(cadena.toUpperCase()))
			.sorted(cmp)
			.map(Articulo::titulo)
			.distinct()
			.toList();
}
```

Ejercicio 4.3 (1 punto)

```
public SortedMap<Extension, Integer> getMaxArticulosPorExtension(
		LocalDate f1, LocalDate f2) {
	Comparator<Revista> cmp = Comparator.comparing(r ->
		r.getArticulos().size());
	return getRevistas().stream()
			.filter(r -> !r.getFechaPublicacion().isBefore(f1)
				&& r.getFechaPublicacion().isBefore(f2))
			.collect(Collectors.groupingBy(
				Revista::getExtension,
				TreeMap::new,
				Collectors.collectingAndThen(
					Collectors.maxBy(cmp),
					o -> o.get().getArticulos().size()
				)
			));
}
```

Ejercicio 4.4 (1 punto)

```
	public Map<String, Double> getPorcentajePaginasPorCategoria() {
		Map<String, Integer> maux = getTotalPaginasPorCategoria(null);

		Integer totalPaginas = maux.values().stream()
				.mapToInt(np -> np)
				.sum();

		return maux.keySet().stream()
				.collect(Collectors.toMap(
						k -> k,
						k -> maux.get(k) * 100. / totalPaginas
				));
	}
```

Ejercicio 4.5 (1,5 puntos)

```
	public List<String> getNumerosArticulosLargos() {

		List<String> res = getRevistas().stream()

				.filter(r -> r.getArticulos().stream()
						.filter(Articulo::articuloLargo)
						.map(Articulo::categoria)
						.distinct()
						.count() >= 2
				)
				.sorted(Comparator.comparing(Revista::getFechaPublicacion))
				.map(Revista::getNumero)
				.collect(Collectors.toList());

		return res;
	}
```

Test

```
public class TestAsociacion {

	public static void main(String[] args) {

		List<Revista> revistas =
			FactoriaRevistas.leeRevistas("data/revistas_informatica.csv");

		AsociacionUniversitaria as =
				new AsociacionUniversitaria("AAA", 2005, "US", revistas);

		System.out.println("\nFactoría    " + "=".repeat(60));
		System.out.println("Número de revistas publicadas por la Asociación: "
			+ revistas.size());

		System.out.println("\nEjercicio 1 " + "=".repeat(60));
		testGetTotalPaginasPorCategoria(as, 2024);
		System.out.println();
		testGetTotalPaginasPorCategoria(as, null);
```

```
        System.out.println("\nEjercicio 2 " + "=".repeat(60));
        testGetTitulosOrdenados(as, "red");

        System.out.println("\nEjercicio 3 " + "=".repeat(60));
        testGetMaxArticulosPorExtension(as, LocalDate.of(2020, 1, 1),
                LocalDate.of(2025, 6, 30));

        System.out.println("\nEjercicio 4 " + "=".repeat(60));
        testGetPorcentajePaginasPorCategoria(as);

        System.out.println("\nEjercicio 5 " + "=".repeat(60));
        testGetNumerosArticulosLargos(as);
    }

    private static void testGetTotalPaginasPorCategoria(
                AsociacionUniversitaria as, Integer a) {
        try {
                Map<String, Integer> res = as.getTotalPaginasPorCategoria(a);
                String msg = String.format("A partir del año %d, " +
                        "el número total de páginas por categoría es:\n%s",
                        a, res);
                System.out.println(msg);
        } catch (Exception e) {
                System.out.println(">>>Capturada excepción inesperada"
                        + e.getMessage());
        }
    }

    private static void testGetTitulosOrdenados(
                AsociacionUniversitaria as, String cad) {
        try {
                List<String> res = as.getTitulosOrdenados(cad);
                String msg = String.format("Títulos ordenados de artículos " +
                        "que contienen la cadena '%s':\n%s",
                        cad, res);
                System.out.println(msg);
        } catch (Exception e) {
                System.out.println(">>>Capturada excepción inesperada"
                        + e.getMessage());
        }
    }

    private static void testGetMaxArticulosPorExtension(
                AsociacionUniversitaria as, LocalDate f1, LocalDate f2) {
        try {
                SortedMap<Extension, Integer> res =
                        as.getMaxArticulosPorExtension(f1, f2);
                String msg = String.format("Entre las fechas " + f1 + " y " + f2 +
                        ", el máximo número de artículos por extensión es:\n%s",
                        res);
                System.out.println(msg);
        } catch (Exception e) {
                System.out.println(">>>Capturada excepción inesperada"
                        + e.getMessage());
        }
    }

    private static void testGetPorcentajePaginasPorCategoria(
                AsociacionUniversitaria as) {
```

```
		try {
			Map<String, Double> res = as.getPorcentajePaginasPorCategoria();
			String msg = String.format("El % de páginas por categoría es:\n%s",
					res);
			System.out.println(msg);
		} catch (Exception e) {
			System.out.println(">>>Capturada excepción inesperada"
					+ e.getMessage());
		}
	}

	private static void testGetNumerosArticulosLargos(
			AsociacionUniversitaria as) {
		try {
			List<String> res = as.getNumerosArticulosLargos();
			String msg = String.format("Los números que tienen al menos " +
				"dos artículos largos de categorías diferentes son:\n%s",
				res);
			System.out.println(msg);
		} catch (Exception e) {
			System.out.println(">>>Capturada excepción inesperada"
					+ e.getMessage());
		}
	}

}
```